6901012610196 Chonpatchut Boonpalit Cpr.E S.2

รายงาน Lab 8

Object-Oriented Programming (OOP)

ข้อ 1 การสร้าง Class Student และการอ่านข้อมูลนักเรียนจาก Text File

1.1 เป้าหมายของโปรแกรม

โปรแกรมนี้มีเป้าหมายเพื่อศึกษาการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming: OOP) โดยสร้าง Class Student สำหรับเก็บข้อมูลนักเรียน ได้แก่ student_id, name, course และ age พร้อมสร้าง Method show() สำหรับแสดงข้อมูลของนักเรียนแต่ละคน

นอกจากนี้ยังฝึกอ่านข้อมูลนักเรียนจาก Text File แล้วนำมาสร้างเป็น Object เก็บไว้ใน List จากนั้นเขียนฟังก์ชันเพื่อค้นหานักเรียนที่มีอายุน้อยที่สุด และแสดงข้อมูลโดยเรียกใช้ Method show()

1.2 ขั้นตอนการทำงาน

1. สร้าง Class Student เพื่อกำหนดข้อมูลของนักเรียน
2. สร้างไฟล์ student.txt สำหรับเก็บข้อมูลนักเรียน
3. อ่านข้อมูลจากไฟล์ทีละบรรทัด
4. แยกข้อมูลแต่ละส่วนออกจากกัน
5. สร้าง Object ของนักเรียนแต่ละคน
6. เก็บ Object ทั้งหมดไว้ใน List
7. ค้นหานักเรียนที่มีอายุน้อยที่สุด
8. แสดงข้อมูลของนักเรียนคนนั้น

1.3 โค้ดโปรแกรม

ไฟล์ student.txt

69001,Ing,Computer Engineering,18

69002,Tawan,Computer Engineering,19

69003,Pim,Electrical Engineering,18

69004,Boat,Computer Engineering,21

69005,Title,Semi-Conductor Engineering,17

69006,Baitong,Computer Engineering,18

69007,Praew,Electrical Engineering,28

69008,Nutjang,Industrial and Electrical Engineering,18

ไฟล์ Python

```
class Student:
    def __init__(self, student_id, name, course, age):
        self.student_id = student_id
        self.name = name
        self.course = course
        self.age = int(age)

    def show(self):
        print("Student ID:", self.student_id)
        print("Name:", self.name)
        print("Course:", self.course)
        print("Age:", self.age)


def read_student():
    students = []

    with open("student.txt", "r") as file:
        for line in file:
            data = line.strip().split(",")

            student = Student(
                data[0], data[1], data[2], data[3]
            )

            students.append(student)

    return students


def show_youngest(students):
    youngest = students[0]

    for student in students:
        if student.age < youngest.age:
            youngest = student

    print("Youngest Student")
    youngest.show()


students = read_student()
```

show_youngest(students)

1.4 อธิบายการทำงานของโค้ด

ส่วนที่ 1 การสร้าง Class Student

```
class Student:
```

ใช้สร้าง Class ชื่อ Student ซึ่งเป็นแม่แบบสำหรับสร้าง Object ของนักเรียน โดยกำหนดข้อมูลและการทำงานที่นักเรียนแต่ละคนต้องมี

```
def __init__(self, student_id, name, course, age):
```

เป็น Constructor ที่ทำงานเมื่อสร้าง Object ใหม่ โดยรับข้อมูลรหัสนักศึกษา ชื่อ สาขาวิชา และอายุเข้ามา

`self` ใช้อ้างถึง Object ที่กำลังทำงานอยู่

```
self.student_id = student_id
```

นำค่ารหัสนักศึกษาที่รับเข้ามาเก็บไว้ใน Attribute student_id ของ Object นั้น

```
self.name = name
```

นำชื่อของนักเรียนมาเก็บไว้ใน Attribute name

```
self.course = course
```

นำชื่อสาขาวิชามาเก็บไว้ใน Attribute course

```
self.age = int(age)
```

นำอายุมาเก็บไว้ใน Attribute age โดยใช้ int() แปลงข้อมูลให้เป็นจำนวนเต็ม เนื่องจากข้อมูลที่อ่านจาก Text File เป็น String

```
def show(self):
```

สร้าง Method show() สำหรับแสดงข้อมูลของนักเรียนแต่ละคน

```
print("Student ID:", self.student_id)
```

แสดงรหัสนักศึกษาของ Object ปัจจุบัน

```
print("Name:", self.name)
```

แสดงชื่อนักเรียน

```
print("Course:", self.course)
```

แสดงสาขาวิชา

```
print("Age:", self.age)
```

แสดงอายุของนักเรียน

ส่วนที่ 2 การอ่านข้อมูลจาก Text File

```
def read_student():
```

สร้างฟังก์ชัน read_student() สำหรับอ่านข้อมูลจากไฟล์และสร้าง Object ของนักเรียน

```
students = []
```

สร้าง List ว่างชื่อ students เพื่อใช้เก็บ Object ของนักเรียนทั้งหมด

```
with open("student.txt", "r") as file:
```

เปิดไฟล์ student.txt ในโหมดอ่านข้อมูล โดยใช้ with เพื่อให้ระบบปิดไฟล์โดยอัตโนมัติเมื่อทำงานเสร็จ

```
for line in file:
```

วนอ่านข้อมูลจากไฟล์ทีละบรรทัด โดยเก็บข้อมูลแต่ละบรรทัดไว้ในตัวแปร line

```
data = line.strip().split(",")
```

ใช้ strip() เพื่อตัดอักขระขึ้นบรรทัดใหม่บริเวณท้ายข้อความ จากนั้นใช้ split(",") แบ่งข้อมูลโดยใช้เครื่องหมายจุลภาค

ตัวอย่างข้อมูล

69001,Ing,Computer Engineering,19

จะถูกแบ่งเป็น List ดังนี้

["69001", "Ing", "Computer Engineering", "19"]

```
student = Student(data[0], data[1], data[2], data[3])
```

นำข้อมูลทั้ง 4 ส่วนมาสร้าง Object ใหม่จาก Class Student โดยเรียงตามลำดับรหัสนักศึกษา ชื่อ สาขาวิชา และอายุ

```
students.append(student)
```

เพิ่ม Object ที่สร้างขึ้นเข้าไปใน List students

```
return students
```

ส่ง List ที่เก็บ Object ของนักเรียนทั้งหมดกลับไปยังส่วนที่เรียกใช้ฟังก์ชัน

ส่วนที่ 3 การค้นหานักเรียนที่อายุน้อยที่สุด

```
def show_youngest(students):
```

สร้างฟังก์ชันสำหรับรับ List ของนักเรียนเข้ามา แล้วค้นหานักเรียนที่มีอายุน้อยที่สุด

```
youngest = students[0]
```

กำหนดให้นักเรียนคนแรกเป็นผู้ที่มีอายุน้อยที่สุดในเบื้องต้น เพื่อใช้เป็นค่าเริ่มต้นในการเปรียบเทียบ

```
for student in students:
```

วนตรวจสอบ Object ของนักเรียนทุกคนใน List

```
if student.age < youngest.age:
```

เปรียบเทียบอายุของนักเรียนคนปัจจุบันกับอายุของนักเรียนที่มีอายุน้อยที่สุดในขณะนั้น

```
youngest = student
```

หากนักเรียนคนปัจจุบันมีอายุน้อยกว่า จะเปลี่ยนให้ Object ของนักเรียนคนนั้นเป็นผู้ที่มีอายุน้อยที่สุดแทน

```
print("Youngest Student")
```

แสดงข้อความแจ้งว่าส่วนข้อมูลต่อไปเป็นข้อมูลของนักเรียนที่อายุน้อยที่สุด

```
youngest.show()
```

เรียกใช้ Method show() ของ Object ที่มีอายุน้อยที่สุด เพื่อแสดงข้อมูลทั้งหมด

ส่วนที่ 4 การเรียกใช้งานโปรแกรม

```
students = read_student()
```

เรียกใช้ฟังก์ชัน read_student() เพื่ออ่านข้อมูลจากไฟล์และสร้าง List ของ Object จากนั้นนำค่าที่ฟังก์ชันส่งกลับมาเก็บไว้ในตัวแปร students

```
show_youngest(students)
```

ส่ง List ของนักเรียนเข้าไปในฟังก์ชัน show_youngest() เพื่อค้นหาและแสดงข้อมูลของนักเรียนที่อายุน้อยที่สุด

1.5 ผลลัพธ์ของโปรแกรม

Youngest Student
Student ID: 69002
Name: Jane
Course: Computer Engineering
Age: 18

จากผลลัพธ์พบว่า Jane เป็นนักเรียนที่มีอายุน้อยที่สุดจากข้อมูลตัวอย่าง โดยมีอายุ 18 ปี

1.6 Error ที่อาจพบและวิธีแก้ไข

1. FileNotFoundError

สาเหตุ: โปรแกรมไม่พบไฟล์ student.txt เนื่องจากไฟล์ไม่ได้อยู่ในตำแหน่งที่กำหนด หรือชื่อไฟล์ไม่ถูกต้อง

วิธีแก้ไข: ตรวจสอบชื่อไฟล์และตำแหน่งที่เก็บไฟล์ โดยในโปรแกรมนี้กำหนดให้ไฟล์ student.txt อยู่ในโฟลเดอร์เดียวกับไฟล์ Python

2. ValueError

สาเหตุ: ข้อมูลอายุไม่สามารถแปลงเป็นจำนวนเต็มด้วย int() ได้ เช่น มีตัวอักษรอยู่ในช่องอายุ

วิธีแก้ไข: ตรวจสอบข้อมูลใน Text File ให้ช่องอายุเป็นตัวเลขเท่านั้น

3. IndexError

สาเหตุ: ไม่มีข้อมูลนักเรียนในไฟล์ ทำให้ไม่สามารถเข้าถึง students[0] ได้

วิธีแก้ไข: ตรวจสอบให้แน่ใจว่าไฟล์มีข้อมูลนักเรียนอย่างน้อยหนึ่งคนก่อนเรียกใช้ฟังก์ชันค้นหา

ข้อ 2 การสร้าง Class Product และการอ่านข้อมูลสินค้าจาก Text File

2.1 เป้าหมายของโปรแกรม

โปรแกรมนี้มีเป้าหมายเพื่อศึกษาการสร้าง Class Product สำหรับเก็บข้อมูลสินค้า ได้แก่ name, price และ quantity พร้อมสร้าง Method show() สำหรับแสดงข้อมูลสินค้า

โปรแกรมสามารถอ่านข้อมูลสินค้าจาก Text File แล้วสร้าง Object ของสินค้าแต่ละรายการ เก็บไว้ใน List จากนั้นค้นหาสินค้าที่มีจำนวนมากที่สุด และคำนวณมูลค่ารวมของสินค้านั้นจากราคา × จำนวนสินค้า

2.2 ขั้นตอนการทำงาน

1. สร้าง Class Product สำหรับเก็บข้อมูลสินค้า
2. สร้างไฟล์ product.txt สำหรับเก็บข้อมูลสินค้า
3. อ่านข้อมูลจากไฟล์ทีละบรรทัด
4. สร้าง Object ของสินค้าแต่ละรายการ
5. เก็บ Object ทั้งหมดไว้ใน List
6. ค้นหาสินค้าที่มีจำนวนมากที่สุด
7. แสดงข้อมูลสินค้าและคำนวณมูลค่ารวม

2.3 โค้ดโปรแกรม

ไฟล์ product.txt

```
Pen,10,20
Book,50,15
Pencil,5,30
Bag,250,5
```

ไฟล์ Python

```
class Product:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = float(price)
        self.quantity = int(quantity)

    def show(self):
        print("Name:", self.name)
        print("Price:", self.price)
        print("Quantity:", self.quantity)


def read_product():
    products = []

    with open("product.txt", "r") as file:
        for line in file:
            data = line.strip().split(",")

            product = Product(
                data[0], data[1], data[2]
            )

            products.append(product)

    return products


def show_most_quantity(products):
    most = products[0]

    for product in products:
        if product.quantity > most.quantity:
            most = product

    most.show()

    total = most.price * most.quantity
    print("Total value:", total)
```

products = read_product()

show_most_quantity(products)

2.4 อธิบายการทำงานของโค้ด

ส่วนที่ 1 การสร้าง Class Product

```
class Product:
```

สร้าง Class ชื่อ Product เพื่อใช้เป็นแม่แบบสำหรับสร้าง Object ของสินค้าแต่ละรายการ

```
def __init__(self, name, price, quantity):
```

สร้าง Constructor สำหรับรับชื่อสินค้า ราคา และจำนวนสินค้าเมื่อสร้าง Object ใหม่

```
self.name = name
```

เก็บชื่อสินค้าไว้ใน Attribute name

```
self.price = float(price)
```

เก็บราคาสินค้าไว้ใน Attribute price โดยใช้ float() แปลงข้อมูลให้เป็นตัวเลขที่สามารถมีทศนิยมได้

```
self.quantity = int(quantity)
```

เก็บจำนวนสินค้าไว้ใน Attribute quantity โดยใช้ int() แปลงข้อมูลให้เป็นจำนวนเต็ม

```
def show(self):
```

สร้าง Method show() สำหรับแสดงข้อมูลสินค้า

```
print("Name:", self.name)
```

แสดงชื่อสินค้า

```
print("Price:", self.price)
```

แสดงราคาสินค้า

```
print("Quantity:", self.quantity)
```

แสดงจำนวนสินค้า

ส่วนที่ 2 การอ่านข้อมูลสินค้า

```
def read_product():
```

สร้างฟังก์ชันสำหรับอ่านข้อมูลสินค้าจากไฟล์

```
products = []
```

สร้าง List ว่างสำหรับเก็บ Object ของสินค้า

```
with open("product.txt", "r") as file:
```

เปิดไฟล์ product.txt ในโหมดอ่านข้อมูล

```
for line in file:
```

วนอ่านข้อมูลจากไฟล์ทีละบรรทัด

```
data = line.strip().split(",")
```

ตัดอักขระขึ้นบรรทัดใหม่และแบ่งข้อมูลสินค้าออกเป็นส่วน ๆ โดยใช้เครื่องหมายจุลภาค

```
product = Product(data[0], data[1], data[2])
```

นำข้อมูลชื่อสินค้า ราคา และจำนวนสินค้า มาสร้าง Object ใหม่จาก Class Product

```
products.append(product)
```

เพิ่ม Object ของสินค้าที่สร้างขึ้นเข้าไปใน List products

```
return products
```

ส่ง List ที่เก็บ Object ของสินค้าทั้งหมดกลับไปยังส่วนที่เรียกใช้ฟังก์ชัน

ส่วนที่ 3 การค้นหาสินค้าที่มีจำนวนมากที่สุด

```
def show_most_quantity(products):
```

สร้างฟังก์ชันสำหรับรับ List ของสินค้า แล้วค้นหาสินค้าที่มีจำนวนมากที่สุด

```
most = products[0]
```

กำหนดให้สินค้าแรกเป็นสินค้าที่มีจำนวนมากที่สุดในเบื้องต้น

```
for product in products:
```

วนตรวจสอบสินค้าทุก Object ใน List

```
if product.quantity > most.quantity:
```

เปรียบเทียบจำนวนสินค้าปัจจุบันกับจำนวนของสินค้าที่มีจำนวนมากที่สุดในขณะนั้น

```
most = product
```

หากสินค้าปัจจุบันมีจำนวนมากกว่า จะเปลี่ยนให้สินค้านั้นเป็นสินค้าที่มีจำนวนมากที่สุด

```
most.show()
```

เรียกใช้ Method show() เพื่อแสดงข้อมูลของสินค้าที่มีจำนวนมากที่สุด

```
total = most.price * most.quantity
```

คำนวณมูลค่ารวมของสินค้า โดยนำราคาต่อชิ้นคูณกับจำนวนสินค้า

```
print("Total value:", total)
```

แสดงมูลค่ารวมของสินค้า

ส่วนที่ 4 การเรียกใช้งานโปรแกรม

```
products = read_product()
```

เรียกใช้ฟังก์ชัน read_product() เพื่ออ่านข้อมูลและสร้าง List ของ Object สินค้า

```
show_most_quantity(products)
```

ส่ง List ของสินค้าเข้าไปในฟังก์ชันเพื่อค้นหาและแสดงข้อมูลสินค้าที่มีจำนวนมากที่สุด

2.5 ผลลัพธ์ของโปรแกรม

Name: Pencil
Price: 5.0
Quantity: 30
Total value: 150.0

จากผลลัพธ์พบว่า Pencil เป็นสินค้าที่มีจำนวนมากที่สุด คือ 30 ชิ้น มีราคาชิ้นละ 5 บาท จึงมีมูลค่ารวมเท่ากับ 150 บาท

2.6 Error ที่อาจพบและวิธีแก้ไข

1. FileNotFoundError

สาเหตุ: โปรแกรมไม่พบไฟล์ product.txt

วิธีแก้ไข: ตรวจสอบชื่อไฟล์และตำแหน่งที่เก็บไฟล์ให้ถูกต้อง

2. ValueError

สาเหตุ: ข้อมูลราคาและจำนวนสินค้าไม่สามารถแปลงเป็นตัวเลขได้ เช่น มีตัวอักษรอยู่ในช่องราคา

วิธีแก้ไข: ตรวจสอบข้อมูลใน Text File ให้ราคาเป็นตัวเลข และจำนวนสินค้าเป็นจำนวนเต็ม

3. IndexError

สาเหตุ: ไม่มีข้อมูลสินค้าในไฟล์ ทำให้ไม่สามารถเข้าถึง products[0] ได้

วิธีแก้ไข: ตรวจสอบให้แน่ใจว่าไฟล์มีข้อมูลสินค้าอย่างน้อยหนึ่งรายการก่อนเรียกใช้ฟังก์ชันค้นหาสินค้า

ข้อ 3 การสร้าง Class สำหรับ Matrix

3.1 เป้าหมายของโปรแกรม

โปรแกรมนี้มีเป้าหมายเพื่อศึกษาการสร้าง Class สำหรับจัดเก็บข้อมูล Matrix และการใช้ Method ในการทำงานกับ Matrix โดย Class จะมี Method จำนวน 4 Method ได้แก่ show() สำหรับแสดง Matrix, add() สำหรับบวก Matrix, subtract() สำหรับลบ Matrix และ multiplication() สำหรับคูณ Matrix

3.2 ขั้นตอนการทำงาน

1. สร้าง Class Matrix
2. กำหนด Attributes สำหรับเก็บข้อมูล Matrix จำนวนแถวและจำนวนคอลัมน์
3. สร้าง Method สำหรับแสดง Matrix
4. สร้าง Method สำหรับบวก Matrix
5. สร้าง Method สำหรับลบ Matrix
6. สร้าง Method สำหรับคูณ Matrix
7. สร้าง Object ของ Matrix จำนวน 2 ตัว
8. เรียกใช้ Method เพื่อแสดงผลการคำนวณ

3.3 โค้ดโปรแกรม

```
class Matrix:
    def __init__(self, data):
        self.data = data
        self.row = len(data)
        self.col = len(data[0])

    def show(self):
        for row in self.data:
            print(row)

    def add(self, other):
        result = []

        for i in range(self.row):
            row = []

            for j in range(self.col):
                value = self.data[i][j] + other.data[i][j]
                row.append(value)

            result.append(row)
```

```
        return Matrix(result)

    def subtract(self, other):
        result = []

        for i in range(self.row):
            row = []

            for j in range(self.col):
                value = self.data[i][j] - other.data[i][j]
                row.append(value)

            result.append(row)

        return Matrix(result)

    def multiplication(self, other):
        result = []

        for i in range(self.row):
            row = []

            for j in range(other.col):
                value = 0

                for k in range(self.col):
                    value += self.data[i][k] * other.data[k][j]

                row.append(value)

            result.append(row)

        return Matrix(result)


a = Matrix([[1, 2], [3, 4]])
b = Matrix([[5, 6], [7, 8]])

print("Matrix A")
a.show()

print("Matrix B")
b.show()

print("A + B")
c = a.add(b)
c.show()
```

```
print("A - B")
d = a.subtract(b)
d.show()

print("A x B")
e = a.multiplication(b)
e.show()
```

3.4 อธิบายการทำงานของโค้ด

ส่วนที่ 1 การสร้าง Class Matrix

```
class Matrix:
```

สร้าง Class ชื่อ Matrix สำหรับเก็บข้อมูลและกำหนดการทำงานของ Matrix

```
def __init__(self, data):
```

สร้าง Constructor สำหรับรับข้อมูล Matrix เข้ามาเก็บไว้ใน Object

```
self.data = data
```

เก็บข้อมูล Matrix ไว้ใน Attribute data ซึ่งเป็น List ซ้อน List

```
self.row = len(data)
```

ใช้ len(data) นับจำนวนแถวของ Matrix แล้วเก็บไว้ใน Attribute row

```
self.col = len(data[0])
```

ใช้ len(data[0]) นับจำนวนสมาชิกในแถวแรก เพื่อหาจำนวนคอลัมน์ของ Matrix

ส่วนที่ 2 Method show()

```
def show(self):
```

สร้าง Method สำหรับแสดงข้อมูล Matrix

```
for row in self.data:
```

วนอ่านข้อมูลทีละแถวจาก Matrix

```
print(row)
```

แสดงข้อมูลของแถวปัจจุบัน

ส่วนที่ 3 Method add()

```
def add(self, other):
```

สร้าง Method สำหรับบวก Matrix โดยรับ Matrix อีกตัวเข้ามาในพารามิเตอร์ other

```
result = []
```

สร้าง List ว่างสำหรับเก็บผลลัพธ์ของ Matrix ที่ได้จากการบวก

```
for i in range(self.row):
```

วนทำงานตามจำนวนแถวของ Matrix ตัวแรก โดยตัวแปร i ใช้ระบุตำแหน่งแถว

```
row = []
```

สร้าง List ว่างสำหรับเก็บข้อมูลของแถวผลลัพธ์ในแต่ละรอบ

```
for j in range(self.col):
```

วนทำงานตามจำนวนคอลัมน์ โดยตัวแปร j ใช้ระบุตำแหน่งคอลัมน์

```
value = self.data[i][j] + other.data[i][j]
```

นำสมาชิกของ Matrix ทั้งสองตัวที่อยู่ในตำแหน่งแถวและคอลัมน์เดียวกันมาบวกกัน

```
row.append(value)
```

เพิ่มผลลัพธ์ที่ได้เข้าไปใน List ของแถวปัจจุบัน

```
result.append(row)
```

เมื่อคำนวณสมาชิกครบทุกคอลัมน์แล้ว จะนำแถวที่คำนวณเสร็จแล้วเพิ่มเข้าไปใน Matrix ผลลัพธ์

```
return Matrix(result)
```

สร้าง Object ใหม่จาก Class Matrix โดยใช้ข้อมูลผลลัพธ์ที่คำนวณได้ แล้วส่งกลับไปยังส่วนที่เรียกใช้ Method

ส่วนที่ 4 Method subtract()

```
def subtract(self, other):
```

สร้าง Method สำหรับลบ Matrix โดยรับ Matrix อีกตัวเข้ามาในพารามิเตอร์ other

```
result = []
```

สร้าง List ว่างสำหรับเก็บผลลัพธ์

```
for i in range(self.row):
```

วนทำงานตามจำนวนแถวของ Matrix ตัวแรก

```
row = []
```

สร้าง List ว่างสำหรับเก็บข้อมูลในแถวผลลัพธ์

```
for j in range(self.col):
```

วนทำงานตามจำนวนคอลัมน์ของ Matrix

```
value = self.data[i][j] - other.data[i][j]
```

นำสมาชิกของ Matrix ตัวแรก ลบด้วยสมาชิกของ Matrix ตัวที่สองในตำแหน่งเดียวกัน

```
row.append(value)
```

เพิ่มผลลัพธ์ลงใน List ของแถวปัจจุบัน

```
result.append(row)
```

เพิ่มแถวที่คำนวณเสร็จแล้วเข้าไปใน Matrix ผลลัพธ์

```
return Matrix(result)
```

สร้าง Object ใหม่จากผลลัพธ์ของการลบ Matrix แล้วส่งกลับไป

ส่วนที่ 5 Method multiplication()

```
def multiplication(self, other):
```

สร้าง Method สำหรับคูณ Matrix โดยรับ Matrix อีกตัวเข้ามาในพารามิเตอร์ other

```
result = []
```

สร้าง List ว่างสำหรับเก็บผลลัพธ์ของการคูณ Matrix

```
for i in range(self.row):
```

วนทำงานตามจำนวนแถวของ Matrix ตัวแรก

```
row = []
```

สร้าง List ว่างสำหรับเก็บข้อมูลในแถวผลลัพธ์

```
for j in range(other.col):
```

วนทำงานตามจำนวนคอลัมน์ของ Matrix ตัวที่สอง

```
value = 0
```

กำหนดค่าเริ่มต้นของผลรวมเป็นศูนย์ เพื่อใช้สะสมผลคูณของสมาชิกในแถวและคอลัมน์

```
for k in range(self.col):
```

วนตามจำนวนคอลัมน์ของ Matrix ตัวแรก เพื่อคำนวณผลคูณของสมาชิกในแถวและคอลัมน์

```
value += self.data[i][k] * other.data[k][j]
```

นำสมาชิกในแถวที่ i ของ Matrix ตัวแรกคูณกับสมาชิกในคอลัมน์ที่ j ของ Matrix ตัวที่สอง แล้วนำผลคูณมาบวกสะสมไว้ใน value

```
row.append(value)
```

เมื่อคำนวณผลลัพธ์ของตำแหน่งนั้นเสร็จแล้ว จะนำค่าที่ได้เพิ่มเข้าไปใน List ของแถวปัจจุบัน

```
result.append(row)
```

เมื่อคำนวณครบทุกคอลัมน์แล้ว จะเพิ่มแถวผลลัพธ์เข้าไปใน Matrix ผลลัพธ์

```
return Matrix(result)
```

สร้าง Object ใหม่จาก Class Matrix โดยใช้ข้อมูลผลลัพธ์ของการคูณ แล้วส่งกลับไป

ส่วนที่ 6 การสร้าง Object และเรียกใช้ Method

```
a = Matrix([[1, 2], [3, 4]])
```

สร้าง Object ชื่อ a จาก Class Matrix โดยกำหนดข้อมูล Matrix ขนาด 2 × 2

```
b = Matrix([[5, 6], [7, 8]])
```

สร้าง Object ชื่อ b จาก Class Matrix โดยกำหนดข้อมูล Matrix อีกชุดหนึ่ง

```
print("Matrix A")
```

แสดงข้อความ Matrix A

```
a.show()
```

เรียก Method show() เพื่อแสดงข้อมูลของ Matrix A

```
print("Matrix B")
```

แสดงข้อความ Matrix B

```
b.show()
```

เรียก Method show() เพื่อแสดงข้อมูลของ Matrix B

```
print("A + B")
```

แสดงข้อความ A + B

```
c = a.add(b)
```

เรียก Method add() เพื่อบวก Matrix A และ Matrix B แล้วเก็บผลลัพธ์ไว้ใน Object c

```
c.show()
```

แสดงผลลัพธ์ของการบวก Matrix

```
print("A - B")
```

แสดงข้อความ A - B

```
d = a.subtract(b)
```

เรียก Method subtract() เพื่อหาผลลัพธ์ของการลบ Matrix แล้วเก็บไว้ใน Object d

```
d.show()
```

แสดงผลลัพธ์ของการลบ Matrix

```
print("A x B")
```

แสดงข้อความ A x B

```
e = a.multiplication(b)
```

เรียก Method multiplication() เพื่อคูณ Matrix A และ Matrix B แล้วเก็บผลลัพธ์ไว้ใน Object e

```
e.show()
```

แสดงผลลัพธ์ของการคูณ Matrix

3.5 ผลลัพธ์ของโปรแกรม

Matrix A
[1, 2]
[3, 4]
Matrix B
[5, 6]
[7, 8]
A + B
[6, 8]
[10, 12]
A - B
[-4, -4]
[-4, -4]
A x B
[19, 22]
[43, 50]

จากผลลัพธ์พบว่าโปรแกรมสามารถแสดง Matrix และคำนวณการบวก การลบ และการคูณ Matrix ได้ตามหลักการทางคณิตศาสตร์

3.6 Error ที่อาจพบและวิธีแก้ไข

1. IndexError

สาเหตุ: เข้าถึงตำแหน่งของ Matrix ที่ไม่มีอยู่ เช่น จำนวนแถวหรือคอลัมน์ไม่ตรงกับตำแหน่งที่ต้องการเข้าถึง

วิธีแก้ไข: ตรวจสอบจำนวนแถวและคอลัมน์ของ Matrix ให้ถูกต้อง และตรวจสอบเงื่อนไขของการบวก ลบ และคูณ Matrix

2. TypeError

สาเหตุ: ข้อมูลใน Matrix เป็นชนิดข้อมูลที่ไม่สามารถนำมาคำนวณทางคณิตศาสตร์ได้ เช่น String

วิธีแก้ไข: ตรวจสอบให้ข้อมูลใน Matrix เป็นตัวเลขก่อนนำมาคำนวณ

3. ผลลัพธ์การคูณ Matrix ไม่ถูกต้อง

สาเหตุ: ใช้ตำแหน่งแถวและคอลัมน์ผิด หรือกำหนดลูปการคำนวณไม่ถูกต้อง

วิธีแก้ไข: ตรวจสอบการเข้าถึงสมาชิกของ Matrix โดยใช้ self.data[i][k] สำหรับสมาชิกในแถวของ Matrix ตัวแรก และ other.data[k][j] สำหรับสมาชิกในคอลัมน์ของ Matrix ตัวที่สอง

ข้อ 4 การแสดงรายชื่อไฟล์ใน Directory และการหาขนาดไฟล์

4.1 เป้าหมายของโปรแกรม

โปรแกรมนี้มีเป้าหมายเพื่อศึกษาการใช้ Library os ในภาษา Python สำหรับจัดการไฟล์และ Directory โดยโปรแกรมสามารถรับตำแหน่ง Directory จากผู้ใช้ แสดงรายชื่อไฟล์ทั้งหมดภายใน Directory คำนวณขนาดรวมของไฟล์ทั้งหมด และกรองให้แสดงเฉพาะไฟล์รูปภาพนามสกุล .jpg, .gif และ .png พร้อมคำนวณขนาดรวมของไฟล์รูปภาพ

4.2 ขั้นตอนการทำงาน

1. นำเข้า Library os
2. รับตำแหน่ง Directory จากผู้ใช้
3. สร้างตัวแปรสำหรับเก็บขนาดไฟล์ทั้งหมดและขนาดไฟล์รูปภาพ
4. อ่านรายชื่อไฟล์และโฟลเดอร์ใน Directory
5. ตรวจสอบว่าแต่ละรายการเป็นไฟล์หรือไม่
6. คำนวณขนาดไฟล์และแสดงรายชื่อไฟล์
7. ตรวจสอบนามสกุลไฟล์รูปภาพ
8. คำนวณขนาดรวมของไฟล์ทั้งหมดและไฟล์รูปภาพ

4.3 โค้ดโปรแกรม

```
import os

directory = input("Enter directory: ")

total_size = 0
image_size = 0

for filename in os.listdir(directory):
    path = os.path.join(directory, filename)

    if os.path.isfile(path):
        size = os.path.getsize(path)
        total_size += size

        print(filename, size, "bytes")

        if filename.lower().endswith((".jpg", ".gif", ".png")):
            image_size += size

print("Total file size:", total_size, "bytes")
print("Total image size:", image_size, "bytes")
```

4.4 อธิบายการทำงานของโค้ด

```
import os
```

นำเข้า Library os ซึ่งมีฟังก์ชันสำหรับจัดการไฟล์และ Directory

```
directory = input("Enter directory: ")
```

รับตำแหน่ง Directory จากผู้ใช้ แล้วเก็บไว้ในตัวแปร directory

```
total_size = 0
```

กำหนดตัวแปรสำหรับเก็บขนาดรวมของไฟล์ทั้งหมด โดยเริ่มต้นที่ศูนย์

```
image_size = 0
```

กำหนดตัวแปรสำหรับเก็บขนาดรวมของไฟล์รูปภาพ โดยเริ่มต้นที่ศูนย์

```
for filename in os.listdir(directory):
```

ใช้ os.listdir() อ่านรายชื่อไฟล์และโฟลเดอร์ทั้งหมดใน Directory ที่กำหนด แล้วใช้ลูป for วนตรวจสอบทีละรายการ

```
path = os.path.join(directory, filename)
```

ใช้ os.path.join() รวมตำแหน่ง Directory กับชื่อไฟล์ เพื่อสร้างตำแหน่งไฟล์ที่สามารถนำไปตรวจสอบได้

```
if os.path.isfile(path):
```

ตรวจสอบว่ารายการที่กำลังตรวจสอบเป็นไฟล์จริงหรือไม่ เพื่อไม่ให้นำโฟลเดอร์มาคำนวณรวมในขนาดไฟล์

```
size = os.path.getsize(path)
```

ใช้ os.path.getsize() หาขนาดของไฟล์ โดยผลลัพธ์มีหน่วยเป็น Byte

```
total_size += size
```

นำขนาดของไฟล์ปัจจุบันมาบวกสะสมไว้ในตัวแปร total_size

```
print(filename, size, "bytes")
```

แสดงชื่อไฟล์ ขนาดไฟล์ และหน่วย Byte

```
if filename.lower().endswith((".jpg", ".gif", ".png")):
```

ตรวจสอบว่าไฟล์ปัจจุบันมีนามสกุลเป็น .jpg, .gif หรือ .png หรือไม่ โดยใช้ lower() เปลี่ยนชื่อไฟล์ให้เป็นตัวพิมพ์เล็กก่อนตรวจสอบ

```
image_size += size
```

หากไฟล์เป็นไฟล์รูปภาพตามนามสกุลที่กำหนด จะนำขนาดไฟล์มาบวกสะสมไว้ในตัวแปร image_size

```
print("Total file size:", total_size, "bytes")
```

แสดงขนาดรวมของไฟล์ทั้งหมดใน Directory

```
print("Total image size:", image_size, "bytes")
```

แสดงขนาดรวมของไฟล์รูปภาพทั้งหมดที่มีนามสกุลตามที่กำหนด

4.5 ผลลัพธ์ของโปรแกรม

ตัวอย่างเมื่อมีไฟล์ใน Directory ดังนี้

cat.jpg
dog.png
document.txt
photo.gif

ผลลัพธ์ตัวอย่าง

cat.jpg 10000 bytes
dog.png 20000 bytes

document.txt 5000 bytes
photo.gif 15000 bytes
Total file size: 50000 bytes
Total image size: 45000 bytes

ตัวเลขขนาดไฟล์ในตัวอย่างเป็นข้อมูลสมมติ โดยขนาดจริงขึ้นอยู่กับไฟล์ที่อยู่ใน Directory ของผู้ใช้

4.6 Error ที่อาจพบและวิธีแก้ไข

1. FileNotFoundError

สาเหตุ: Directory ที่ระบุไม่มีอยู่จริง หรือระบุตำแหน่งไม่ถูกต้อง

วิธีแก้ไข: ตรวจสอบตำแหน่ง Directory ให้ถูกต้อง และตรวจสอบว่ามีโฟลเดอร์ดังกล่าวอยู่จริง

2. NotADirectoryError

สาเหตุ: ตำแหน่งที่ระบุเป็นไฟล์ ไม่ใช่ Directory

วิธีแก้ไข: ระบุตำแหน่งของโฟลเดอร์ที่ต้องการตรวจสอบแทนตำแหน่งของไฟล์

3. PermissionError

สาเหตุ: โปรแกรมไม่มีสิทธิ์เข้าถึง Directory หรือไฟล์บางรายการ

วิธีแก้ไข: เลือก Directory ที่สามารถเข้าถึงได้ และตรวจสอบสิทธิ์ในการอ่านไฟล์

ข้อ 5 การวาดลูกโป่ง 5 ลูกด้วย List และ Function ใน Processing

5.1 เป้าหมายของโปรแกรม

โปรแกรมนี้มีเป้าหมายเพื่อศึกษาการใช้ List สำหรับเก็บข้อมูลของลูกโป่งหลายลูก และการสร้างฟังก์ชัน draw_balloon() สำหรับวาดลูกโป่งหนึ่งลูก โดยใช้ Python-mode Processing

นอกจากนี้ยังได้ศึกษาการสร้าง Animation โดยเปลี่ยนตำแหน่งของลูกโป่งในแต่ละ Frame เพื่อให้ลูกโป่งลอยขึ้น และกำหนดให้ลูกโป่งกลับมาเริ่มต้นจากด้านล่างเมื่อเคลื่อนที่พ้นหน้าจอ

5.2 ขั้นตอนการทำงาน

1. สร้าง List สำหรับเก็บข้อมูลลูกโป่ง 5 ลูก
2. กำหนดตำแหน่งและขนาดของลูกโป่งแต่ละลูก
3. สร้างฟังก์ชัน setup() สำหรับกำหนดขนาดหน้าจอ
4. สร้างฟังก์ชัน draw_balloon() สำหรับวาดลูกโป่งหนึ่งลูก
5. สร้างฟังก์ชัน draw() สำหรับวาดลูกโป่งทั้งหมด
6. เปลี่ยนตำแหน่ง y ของลูกโป่งในแต่ละ Frame เพื่อให้ลูกโป่งลอยขึ้น

7. เมื่อลูกโป่งลอยพ้นหน้าจอ ให้กลับมาเริ่มต้นจากด้านล่าง

5.3 โค้ดโปรแกรม

```
balloons = [
    [100, 400, 40, 60],
    [200, 300, 50, 70],
    [300, 450, 45, 65],
    [400, 350, 55, 75],
    [500, 250, 40, 60]
]


def setup():
    size(600, 600)


def draw_balloon(x, y, w, h):
    fill(255, 100, 150)
    ellipse(x, y, w, h)

    stroke(0)
    line(x, y + h / 2, x, y + h / 2 + 40)


def draw():
    background(220)

    for balloon in balloons:
        x = balloon[0]
        y = balloon[1]
        w = balloon[2]
        h = balloon[3]

        draw_balloon(x, y, w, h)

        balloon[1] -= 1

        if balloon[1] < -h:
            balloon[1] = height
```

5.4 อธิบายการทำงานของโค้ด

ส่วนที่ 1 การสร้าง List สำหรับเก็บข้อมูลลูกโป่ง

```
balloons = [
```

สร้าง List ชื่อ balloons เพื่อเก็บข้อมูลของลูกโป่งทั้ง 5 ลูก

```
[100, 400, 40, 60]
```

เก็บข้อมูลของลูกโป่งลูกที่ 1 โดยประกอบด้วยตำแหน่ง x ตำแหน่ง y ความกว้าง และความสูง ตามลำดับ

```
[200, 300, 50, 70]
```

เก็บข้อมูลของลูกโป่งลูกที่ 2

```
[300, 450, 45, 65]
```

เก็บข้อมูลของลูกโป่งลูกที่ 3

```
[400, 350, 55, 75]
```

เก็บข้อมูลของลูกโป่งลูกที่ 4

```
[500, 250, 40, 60]
```

เก็บข้อมูลของลูกโป่งลูกที่ 5

```
]
```

ปิด List ที่ใช้เก็บข้อมูลลูกโป่งทั้ง 5 ลูก

ส่วนที่ 2 การกำหนดขนาดหน้าจอ

```
def setup():
```

สร้างฟังก์ชัน setup() ซึ่ง Processing เรียกใช้งานเมื่อเริ่มต้นโปรแกรม

```
size(600, 600)
```

กำหนดขนาดหน้าจอให้มีความกว้าง 600 พิกเซล และความสูง 600 พิกเซล

ส่วนที่ 3 การสร้างฟังก์ชันวาดลูกโป่ง

```
def draw_balloon(x, y, w, h):
```

สร้างฟังก์ชัน draw_balloon() สำหรับวาดลูกโป่งหนึ่งลูก โดยรับตำแหน่ง x, y และขนาดความกว้างกับความสูงเข้ามา

```
fill(255, 100, 150)
```

กำหนดสีที่จะใช้เติมภายในลูกโป่ง โดยใช้ค่าสี RGB

```
ellipse(x, y, w, h)
```

วาดรูปวงรีเพื่อแทนตัวลูกโป่ง โดยกำหนดตำแหน่งและขนาดจากพารามิเตอร์ที่รับเข้ามา

```
stroke(0)
```

กำหนดสีเส้นขอบของวัตถุเป็นสีดำ

```
line(x, y + h / 2, x, y + h / 2 + 40)
```

วาดเส้นเชือกของลูกโป่ง โดยกำหนดจุดเริ่มต้นบริเวณด้านล่างของวงรี และลากเส้นลงไปอีก 40 พิกเซล

ส่วนที่ 4 การวาดลูกโป่งและสร้าง Animation

```
def draw():
```

สร้างฟังก์ชัน draw() ซึ่ง Processing จะเรียกทำงานซ้ำอย่างต่อเนื่อง เพื่อสร้างภาพเคลื่อนไหว

```
background(220)
```

กำหนดสีพื้นหลังของหน้าจอเป็นสีเทา โดยใช้ค่าสีเทา 220

```
for balloon in balloons:
```

วนทำงานกับข้อมูลลูกโป่งแต่ละลูกที่อยู่ใน List balloons

```
x = balloon[0]
```

ดึงค่าตำแหน่ง x ของลูกโป่งออกมาเก็บไว้ในตัวแปร x

```
y = balloon[1]
```

ดึงค่าตำแหน่ง y ของลูกโป่งออกมาเก็บไว้ในตัวแปร y

```
w = balloon[2]
```

ดึงค่าความกว้างของลูกโป่งออกมาเก็บไว้ในตัวแปร w

```
h = balloon[3]
```

ดึงค่าความสูงของลูกโป่งออกมาเก็บไว้ในตัวแปร h

```
draw_balloon(x, y, w, h)
```

เรียกใช้ฟังก์ชัน draw_balloon() เพื่อวาดลูกโป่งตามข้อมูลที่ดึงออกมาจาก List

```
balloon[1] -= 1
```

ลดค่าตำแหน่ง y ของลูกโป่งลง 1 ในแต่ละ Frame ทำให้ลูกโป่งเคลื่อนที่ขึ้นด้านบน เนื่องจากใน Processing ตำแหน่ง y ที่น้อยลงหมายถึงตำแหน่งที่อยู่สูงขึ้น

```
if balloon[1] < -h:
```

ตรวจสอบว่าลูกโป่งเคลื่อนที่พ้นขอบด้านบนของหน้าจอแล้วหรือไม่

```
balloon[1] = height
```

หากลูกโป่งพ้นหน้าจอแล้ว จะกำหนดตำแหน่ง y ให้กลับมาอยู่บริเวณด้านล่างของหน้าจอ เพื่อให้ลูกโป่งสามารถลอยขึ้นได้อีกครั้ง

5.5 ผลลัพธ์ของโปรแกรม

เมื่อรันโปรแกรม จะปรากฏหน้าจอขนาด 600 × 600 พิกเซล และมีลูกโป่งจำนวน 5 ลูกที่มีขนาดและตำแหน่งแตกต่างกัน ลูกโป่งแต่ละลูกจะเคลื่อนที่ขึ้นด้านบนอย่างต่อเนื่อง เมื่อพ้นขอบด้านบนของหน้าจอจะกลับมาเริ่มต้นจากด้านล่างอีกครั้ง

5.6 Error ที่อาจพบและวิธีแก้ไข

1. NameError

สาเหตุ: ใช้ตัวแปรที่ยังไม่ได้ประกาศ หรือสะกดชื่อตัวแปรไม่ตรงกับที่กำหนดไว้

วิธีแก้ไข: ตรวจสอบชื่อตัวแปรให้ถูกต้อง และตรวจสอบว่ามีการประกาศตัวแปรก่อนนำไปใช้งาน

2. TypeError

สาเหตุ: ส่งจำนวนอาร์กิวเมนต์ไม่ตรงกับที่ฟังก์ชันกำหนด เช่น เรียก draw_balloon() โดยส่งข้อมูลไม่ครบ 4 ค่า

วิธีแก้ไข: ตรวจสอบการเรียกใช้ฟังก์ชันให้ส่งข้อมูลตำแหน่งและขนาดของลูกโป่งครบทั้ง 4 ค่า

3. ลูกโป่งไม่เคลื่อนที่

สาเหตุ: ไม่ได้เปลี่ยนค่าตำแหน่ง y ในแต่ละ Frame หรือเขียนคำสั่งเปลี่ยนตำแหน่งไว้ผิดตำแหน่ง

วิธีแก้ไข: ตรวจสอบให้มีคำสั่ง balloon[1] -= 1 อยู่ภายในลูปที่ทำงานในฟังก์ชัน draw()

ข้อ 6 การสร้าง Class Balloon และ Animation ใน Processing

6.1 เป้าหมายของโปรแกรม

โปรแกรมนี้มีเป้าหมายเพื่อศึกษาการนำแนวคิด OOP มาใช้กับการสร้าง Animation โดยสร้าง Class Balloon สำหรับเก็บข้อมูลของลูกโป่งหนึ่งลูก ได้แก่ ตำแหน่ง x ตำแหน่ง y ความกว้าง และความสูง พร้อมสร้าง Method show() สำหรับวาดลูกโป่ง และ Method move() สำหรับควบคุมการเคลื่อนที่

จากนั้นสร้าง Object ของลูกโป่งจำนวน 5 ลูก และเก็บไว้ใน List เพื่อให้ลูกโป่งแต่ละลูกสามารถวาดตัวเองและเคลื่อนที่ขึ้นด้านบนได้

6.2 ขั้นตอนการทำงาน

1. สร้าง Class Balloon
2. กำหนด Attributes สำหรับเก็บตำแหน่งและขนาดของลูกโป่ง
3. สร้าง Method show() สำหรับวาดลูกโป่ง
4. สร้าง Method move() สำหรับควบคุมการเคลื่อนที่
5. สร้าง Object ของลูกโป่งจำนวน 5 ลูก
6. เก็บ Object ทั้งหมดไว้ใน List
7. วาดและเคลื่อนที่ลูกโป่งทุกลูกในฟังก์ชัน draw()
8. เมื่อลูกโป่งพ้นหน้าจอ ให้กลับมาเริ่มต้นจากด้านล่าง

6.3 โค้ดโปรแกรม

```
class Balloon:
    def __init__(self, x, y, w, h):
        self.x = x
        self.y = y
        self.w = w
        self.h = h

    def show(self):
        fill(255, 100, 150)
        ellipse(self.x, self.y, self.w, self.h)

        stroke(0)
        line(
            self.x,
            self.y + self.h / 2,
            self.x,
            self.y + self.h / 2 + 40
        )

    def move(self):
        self.y -= 1

        if self.y < -self.h:
            self.y = height


balloons = [
    Balloon(100, 400, 40, 60),
    Balloon(200, 300, 50, 70),
    Balloon(300, 450, 45, 65),
    Balloon(400, 350, 55, 75),
    Balloon(500, 250, 40, 60)
]
```

```
def setup():
    size(600, 600)


def draw():
    background(220)

    for balloon in balloons:
        balloon.show()
        balloon.move()
```

6.4 อธิบายการทำงานของโค้ด

ส่วนที่ 1 การสร้าง Class Balloon

```
class Balloon:
```

สร้าง Class ชื่อ Balloon เพื่อใช้เป็นแม่แบบสำหรับสร้าง Object ของลูกโป่งแต่ละลูก

```
def __init__(self, x, y, w, h):
```

สร้าง Constructor สำหรับรับตำแหน่งและขนาดของลูกโป่งเมื่อสร้าง Object ใหม่

```
self.x = x
```

เก็บตำแหน่งแนวนอนของลูกโป่งไว้ใน Attribute x

```
self.y = y
```

เก็บตำแหน่งแนวตั้งของลูกโป่งไว้ใน Attribute y

```
self.w = w
```

เก็บความกว้างของลูกโป่งไว้ใน Attribute w

```
self.h = h
```

เก็บความสูงของลูกโป่งไว้ใน Attribute h

ส่วนที่ 2 Method show()

```
def show(self):
```

สร้าง Method show() สำหรับวาดลูกโป่งตามข้อมูลที่เก็บไว้ใน Attributes ของ Object

```
fill(255, 100, 150)
```

กำหนดสีภายในลูกโป่งเป็นสีชมพู

```
ellipse(self.x, self.y, self.w, self.h)
```

วาดวงรีแทนตัวลูกโป่ง โดยใช้ตำแหน่งและขนาดที่เก็บไว้ใน Object ปัจจุบัน

```
stroke(0)
```

กำหนดสีเส้นขอบของวัตถุเป็นสีดำ

```
line(...)
```

ใช้วาดเส้นเชือกของลูกโป่ง โดยรับพิกัดจุดเริ่มต้นและจุดสิ้นสุดของเส้น

```
self.x
```

กำหนดตำแหน่ง x ของจุดเริ่มต้นเส้นเชือก

```
self.y + self.h / 2
```

กำหนดตำแหน่ง y ของจุดเริ่มต้นเส้นเชือก ซึ่งอยู่บริเวณด้านล่างของวงรี

```
self.x
```

กำหนดตำแหน่ง x ของจุดสิ้นสุดเส้นเชือกให้ตรงกับจุดเริ่มต้น

```
self.y + self.h / 2 + 40
```

กำหนดตำแหน่ง y ของจุดสิ้นสุดเส้นเชือก โดยลากลงไปอีก 40 พิกเซล

ส่วนที่ 3 Method move()

```
def move(self):
```

สร้าง Method move() สำหรับควบคุมการเคลื่อนที่ของลูกโป่งแต่ละ Object

```
self.y -= 1
```

ลดตำแหน่ง y ลง 1 ทำให้ลูกโป่งเคลื่อนที่ขึ้นด้านบนในแต่ละ Frame

```
if self.y < -self.h:
```

ตรวจสอบว่าลูกโป่งเคลื่อนที่พ้นขอบด้านบนของหน้าจอแล้วหรือไม่

```
self.y = height
```

หากลูกโป่งพ้นหน้าจอแล้ว จะกำหนดตำแหน่ง y ให้กลับมาอยู่ด้านล่างของหน้าจออีกครั้ง

ส่วนที่ 4 การสร้าง Object ของลูกโป่ง

```
balloons = [
```

สร้าง List ชื่อ balloons เพื่อเก็บ Object ของลูกโป่งทั้งหมด

```
Balloon(100, 400, 40, 60)
```

สร้าง Object ของลูกโป่งลูกที่ 1 โดยกำหนดตำแหน่ง x, y และขนาด

```
Balloon(200, 300, 50, 70)
```

สร้าง Object ของลูกโป่งลูกที่ 2

```
Balloon(300, 450, 45, 65)
```

สร้าง Object ของลูกโป่งลูกที่ 3

```
Balloon(400, 350, 55, 75)
```

สร้าง Object ของลูกโป่งลูกที่ 4

```
Balloon(500, 250, 40, 60)
```

สร้าง Object ของลูกโป่งลูกที่ 5

```
]
```

ปิด List ที่เก็บ Object ของลูกโป่งทั้ง 5 ลูก

ส่วนที่ 5 การกำหนดขนาดหน้าจอ

```
def setup():
```

สร้างฟังก์ชัน setup() ซึ่ง Processing จะเรียกใช้เมื่อเริ่มต้นโปรแกรม

```
size(600, 600)
```

กำหนดขนาดหน้าจอให้มีความกว้าง 600 พิกเซล และความสูง 600 พิกเซล

ส่วนที่ 6 การวาดและเคลื่อนที่ลูกโป่ง

```
def draw():
```

สร้างฟังก์ชัน draw() สำหรับวาดภาพและสร้าง Animation โดย Processing จะเรียกฟังก์ชันนี้ซ้ำอย่างต่อเนื่อง

```
background(220)
```

กำหนดสีพื้นหลังของหน้าจอเป็นสีเทา

```
for balloon in balloons:
```

วนทำงานกับ Object ของลูกโป่งทุกลูกที่อยู่ใน List balloons

`balloon.show()`

เรียกใช้ Method show() ของ Object แต่ละลูก เพื่อวาดลูกโป่งตามข้อมูลของ Object นั้น

`balloon.move()`

เรียกใช้ Method move() ของ Object แต่ละลูก เพื่อเปลี่ยนตำแหน่ง y ให้ลูกโป่งลอยขึ้น

6.5 ผลลัพธ์ของโปรแกรม

เมื่อรันโปรแกรม จะปรากฏลูกโป่งจำนวน 5 ลูกบนหน้าจอ โดยลูกโป่งแต่ละลูกมีตำแหน่งและขนาดแตกต่างกัน ลูกโป่งจะลอยขึ้นด้านบนอย่างต่อเนื่อง และกลับมาเริ่มต้นจากด้านล่างเมื่อเคลื่อนที่พ้นหน้าจอ

6.6 Error ที่อาจพบและวิธีแก้ไข

1. AttributeError

สาเหตุ: เรียกใช้ Attribute หรือ Method ที่ไม่มีอยู่ใน Class เช่น ใช้ balloon.width ทั้งที่กำหนดชื่อ Attribute เป็น w

วิธีแก้ไข: ตรวจสอบชื่อ Attribute และ Method ให้ตรงกับที่กำหนดไว้ใน Class Balloon

2. TypeError

สาเหตุ: สร้าง Object โดยส่งจำนวนข้อมูลไม่ตรงกับ Constructor เช่น ส่งข้อมูลเพียง 3 ค่า ทั้งที่ Constructor ต้องการ 4 ค่า

วิธีแก้ไข: ตรวจสอบการสร้าง Object ให้ส่งข้อมูลตำแหน่ง x, y และขนาด w, h ครบทั้ง 4 ค่า

3. ลูกโป่งไม่ลอยขึ้น

สาเหตุ: ไม่ได้เรียกใช้ Method move() ในฟังก์ชัน draw() หรือไม่ได้เปลี่ยนค่าตำแหน่ง y

วิธีแก้ไข: ตรวจสอบให้มีคำสั่ง balloon.move() ภายในลูปที่วนทำงานกับลูกโป่งทุกลูก

ข้อ 7 การเปรียบเทียบการใช้ Class กับการไม่ใช้ Class

7.1 เป้าหมายของการเปรียบเทียบ

เพื่อศึกษาความแตกต่างระหว่างการเก็บข้อมูลลูกโป่งในรูปแบบ List และการใช้ Class ในการเก็บข้อมูลลูกโป่ง โดยพิจารณาจากวิธีการเก็บข้อมูล การเรียกใช้งาน การสร้าง Animation และความสะดวกในการจัดการโปรแกรม

7.2 ความแตกต่างระหว่างการใช้ Class และไม่ใช้ Class

การไม่ใช้ Class

การไม่ใช้ Class จะเก็บข้อมูลลูกโป่งแต่ละลูกไว้ใน List โดยใช้ตำแหน่งของสมาชิกใน List เพื่ออ้างอิงข้อมูล เช่น balloon[0] สำหรับตำแหน่ง x และ balloon[1] สำหรับตำแหน่ง y

การวาดลูกโป่งและการเคลื่อนที่จะต้องเขียนแยกไว้ในฟังก์ชัน เช่น draw_balloon() และเขียนคำสั่งเปลี่ยนตำแหน่งของลูกโป่งภายในฟังก์ชัน draw()

วิธีนี้เหมาะกับโปรแกรมที่มีข้อมูลไม่ซับซ้อนและมีการทำงานไม่มาก

การใช้ Class

การใช้ Class จะสร้างแม่แบบสำหรับลูกโป่งหนึ่งลูก โดยเก็บข้อมูลตำแหน่งและขนาดไว้ใน Attributes และกำหนดการทำงานไว้ใน Methods เช่น show() และ move()

เมื่อสร้าง Object ของลูกโป่งแต่ละลูก จะสามารถเรียกใช้ Method เพื่อวาดและเคลื่อนที่ลูกโป่งได้โดยตรง

วิธีนี้ช่วยให้ข้อมูลและการทำงานของลูกโป่งแต่ละลูกอยู่ด้วยกัน ทำให้โปรแกรมอ่านง่ายและจัดการได้สะดวกเมื่อมี Object จำนวนมาก

7.3 สรุปผลการเปรียบเทียบ

จากการทดลองพบว่า การใช้ List โดยไม่ใช้ Class สามารถสร้าง Animation ของลูกโป่งได้เช่นเดียวกับการใช้ Class แต่ต้องอ้างอิงข้อมูลผ่านตำแหน่งของสมาชิกใน List ซึ่งอาจทำให้เกิดความสับสนเมื่อข้อมูลมีหลายรายการ

ส่วนการใช้ Class สามารถจัดเก็บข้อมูลและการทำงานของลูกโป่งไว้ใน Object เดียวกัน ทำให้เรียกใช้งานได้สะดวกและลดความซ้ำซ้อนของโค้ด

ดังนั้น การเลือกใช้ Class หรือไม่ใช้ Class ขึ้นอยู่กับลักษณะของโปรแกรม หากเป็นโปรแกรมขนาดเล็กที่มีข้อมูลไม่ซับซ้อน การใช้ List และ Function ก็สามารถทำงานได้ แต่หากโปรแกรมมี Object หลายตัวและแต่ละตัวมีข้อมูลกับการทำงานของตัวเอง การใช้ Class จะช่วยให้จัดการโปรแกรมได้ง่ายขึ้น

ข้อ 8 สรุป Error ที่พบและวิธีแก้ไขจาก Lab 8

จากการเขียนโปรแกรมใน Lab 8 สามารถพบ Error ที่อาจเกิดจากการสร้าง Class การอ่านข้อมูลจาก Text File การจัดการ List การคำนวณ Matrix และการสร้าง Animation ใน Processing โดย Error แต่ละประเภทมีสาเหตุและวิธีแก้ไขแตกต่างกัน ดังนี้

1. FileNotFoundError

สาเหตุ: ไม่พบไฟล์ที่ต้องการอ่าน

วิธีแก้ไข: ตรวจสอบชื่อไฟล์และตำแหน่งไฟล์ให้ถูกต้อง

2. ValueError

สาเหตุ: แปลงข้อมูลเป็นตัวเลขไม่ได้

วิธีแก้ไข: ตรวจสอบข้อมูลใน Text File ให้เป็นชนิดข้อมูลที่ถูกต้อง

3. IndexError

สาเหตุ: เข้าถึงตำแหน่ง List ที่ไม่มีอยู่

วิธีแก้ไข: ตรวจสอบจำนวนสมาชิกและตำแหน่งที่เข้าถึง

4. AttributeError

สาเหตุ: เรียกใช้ Attribute หรือ Method ที่ไม่มีอยู่

วิธีแก้ไข: ตรวจสอบชื่อ Attribute และ Method

5. TypeError

สาเหตุ: ใช้ข้อมูลผิดชนิดหรือส่งอาร์กิวเมนต์ไม่ครบ

วิธีแก้ไข: ตรวจสอบชนิดข้อมูลและจำนวนอาร์กิวเมนต์

6. NameError

สาเหตุ: ใช้ตัวแปรที่ยังไม่ได้ประกาศ

วิธีแก้ไข: ตรวจสอบชื่อตัวแปรและการประกาศตัวแปร

7. PermissionError

สาเหตุ: ไม่มีสิทธิ์เข้าถึงไฟล์หรือ Directory

วิธีแก้ไข: ตรวจสอบสิทธิ์การเข้าถึง

8. NotADirectoryError

สาเหตุ: ระบุตำแหน่งไฟล์แทน Directory

วิธีแก้ไข: ระบุตำแหน่งโฟลเดอร์ให้ถูกต้อง

การแก้ไข Error ทำให้เข้าใจการทำงานของโปรแกรมมากขึ้น โดยเฉพาะการตรวจสอบชนิดข้อมูล การเข้าถึงข้อมูลใน List การเรียกใช้ Method และการกำหนดตำแหน่งไฟล์ให้ถูกต้อง

ข้อ 9 สรุปสิ่งที่ได้เรียนรู้จาก Lab 8

จากการทำ Lab 8 ทำให้ได้เรียนรู้แนวคิดพื้นฐานของ Object-Oriented Programming หรือ OOP ซึ่งเป็นการเขียนโปรแกรมโดยใช้ Class และ Object โดย Class ทำหน้าที่เป็นแม่แบบสำหรับสร้าง Object และ Object แต่ละตัวสามารถมี Attributes สำหรับเก็บข้อมูล และ Methods สำหรับกำหนดการทำงานของตัวเองได้

นอกจากนี้ยังได้ฝึกสร้าง Class สำหรับเก็บข้อมูลนักเรียนและสินค้า รวมถึงการอ่านข้อมูลจาก Text File แล้วนำมาสร้าง Object และเก็บไว้ใน List จากนั้นนำข้อมูลมาใช้ค้นหานักเรียนที่มีอายุน้อยที่สุด และสินค้าที่มีจำนวนมากที่สุด

ในส่วนของ Matrix ได้เรียนรู้การสร้าง Class สำหรับเก็บข้อมูลแบบ 2 มิติ และการใช้ Method เพื่อแสดงผล บวก ลบ และคูณ Matrix ซึ่งช่วยให้เข้าใจการนำแนวคิด OOP มาใช้กับการคำนวณทางคณิตศาสตร์

ในส่วนของการจัดการไฟล์ ได้เรียนรู้การใช้ Library os สำหรับอ่านรายชื่อไฟล์ใน Directory การหาขนาดไฟล์ และการกรองเฉพาะไฟล์รูปภาพตามนามสกุลที่กำหนด

สุดท้ายได้เรียนรู้การสร้าง Animation ใน Processing ทั้งแบบใช้ Class และไม่ใช้ Class โดยพบว่าทั้งสองวิธีสามารถสร้างผลลัพธ์ได้เหมือนกัน แต่การใช้ Class ช่วยให้การจัดเก็บข้อมูลและการกำหนดการทำงานของ Object แต่ละตัวเป็นระเบียบและสะดวกต่อการแก้ไขมากขึ้น

ข้อ 10 โจทย์ที่คิดว่าน่าสนใจสำหรับเพิ่มใน Lab พร้อมเหตุผล

โจทย์ที่คิดว่าน่าสนใจสำหรับเพิ่มใน Lab คือการสร้างเกมเก็บเหรียญโดยใช้ Class และ Object เนื่องจากเป็นโจทย์ที่สามารถนำความรู้จาก Lab 8 มาประยุกต์ใช้ได้หลายส่วน

โดยสามารถสร้าง Class Player สำหรับเก็บข้อมูลผู้เล่น เช่น ตำแหน่งและคะแนน และสร้าง Class Coin สำหรับเก็บข้อมูลเหรียญแต่ละเหรียญ จากนั้นสร้าง Object ของผู้เล่นและเหรียญหลายตัว แล้วใช้ Method สำหรับวาดและควบคุมการเคลื่อนที่

เหตุผลที่เลือกโจทย์นี้เพราะเป็นการนำแนวคิด OOP ไปใช้กับโปรแกรมที่มีการโต้ตอบและมี Object หลายชนิด ทำให้ได้ฝึกการสร้าง Class การเก็บข้อมูลใน Object การใช้ List การเรียกใช้ Method และการตรวจสอบเงื่อนไขต่าง ๆ ภายในโปรแกรม

นอกจากนี้ยังสามารถพัฒนาเพิ่มเติมได้ เช่น การเพิ่มจำนวนเหรียญ การเพิ่มคะแนนเมื่อผู้เล่นเก็บเหรียญ หรือการเพิ่มสิ่งกีดขวาง เพื่อให้ได้ฝึกเขียนโปรแกรมที่มีความซับซ้อนมากขึ้นอย่างเป็นขั้นตอน